Chapter 15. Service Meshes

Perhaps second only to containers, the term *service mesh* has become synonymous with cloud native development. However, just like containers, service mesh is a broad term that encompasses a variety of open source projects as well as commercial products. Understanding the general role of a service mesh in a cloud native architecture is useful. This chapter will show you what a service mesh is, how different software projects implement them, and finally (and most importantly) when it makes sense to incorporate a service mesh, versus a less complex architecture, into your application.

NOTE

In many abstract cloud native architecture diagrams, it seems that a service mesh is necessary for a cloud native architecture. This is very much not true. When considering adopting a service mesh, you have to balance the complexity of adding a new component (generally provided by a third party) to your list of dependencies. In many cases, it is easier and more reliable to simply depend on the existing Kubernetes resources, if they meet the needs of your application.

We have previously discussed other networking primitives in Kubernetes like Services and Ingress. Given the presence of these networking capabilities in the core of Kubernetes, why is there a need to inject additional capabilities (and complexities) into the networking layer? Fundamentally it comes down to the needs of the software application that is using these networking primitives.

Networking in the core of Kubernetes is really only aware of the application as a destination. Both Service and Ingress resources have label selectors that route traffic to a particular set of Pods, but beyond that there is comparatively little in the way of additional capabilities that these resources bring. As an HTTP load balancer, Ingress goes a little beyond this, but the challenge of defining a common API that fits a wide variety of different existing implementations limits the capabilities in the Ingress API. How can a truly "cloud native" HTTP-routing API be compatible with load balancers and proxies ranging from bare-metal networking devices through to public cloud APIs that were built without thinking about cloud native development?

In a very real way, the development of service mesh APIs outside the core of Kubernetes is a result of this challenge. The Ingress APIs bring HTTP(S) traffic from the outside world into a cloud native application. Within a cloud native application in Kubernetes, freed of the need to be compatible with existing infrastructure, the service mesh APIs provide additional cloud native networking capabilities. So what are these capabilities? There are three general capabilities provided by most service mesh implementations: network encryption and authorization, traffic shaping, and observability. The following sections look at each of these in turn.

Encryption and Authentication with Mutal TLS

Encryption of network traffic between Pods is a key component to security in a microservice architecture. Encryptions provided by Mutual Transport Layer Security, or *mTLS*, is one of the most popular use cases for a service mesh. While it is possible for developers to implement this encryption themselves, certificate handling and traffic encryption is complicated and hard to get right. Leaving the implementation of encryption to individual development teams leads to developers forgetting to add encryption at all, or doing it poorly. When poorly implemented, encryption can negatively impact both reliability and, in the worst case, provide no real security. By contrast, installing a service mesh on your Kubernetes cluster automatically provides encryption to network traffic between every Pod in the cluster. The service mesh adds a sidecar container to every Pod, which transparently intercepts all network communication. In addition to securing the communication, mTLS adds identity to the encryption using client certificates so your application securely knows the identity of every network client.

Traffic Shaping

When you first think about your application design, it is typically a clean diagram with a single box for each microservice or layer in the system (e.g., the frontend service, user preferences service, etc). When implemented in practice, there are actually often multiple instances of any particular microservice running within the application. For example, when you are doing a rollout from version X of your service to version Y, there is a point in the middle of the rollout when you are simultaneously running two different versions of that service. While the middle of a rollout is a temporary state, there are many times when you need to create a longer-running experiment that persists for an extended period. A common model used in the software industry is “dog-fooding” your own software, meaning that a new version of the software is tried internally before anywhere

else. In a dog-fooding model, you may run version Y of your service for a day to a week (or longer) for a subset of users before you roll it out broadly to your full set of users.

Such experiments require the ability to do *traffic shaping*, or routing of requests to different service implementations based on the characteristics of the request. In this example, you would create an experiment where all traffic from your company's internal users went to service Y, while all traffic from the rest of the world still went to service X.

Experiments are useful for a variety of scenarios, including development, where a programmer can send a limited set (typically 1% or less) of real-world traffic to an experimental backend, or you can run an A/B experiment where 50% of users get one experience and 50% of users get another, so that you can build statistical models of which approach is more effective. Experiments are an incredibly useful way to add reliability, agility, and insight to your application, but they are often difficult to implement in practice and thus not used as often as they might otherwise be.

Service meshes change this by building experimentation into the mesh itself. Instead of writing code to implement your experiment, or deploying an entirely new copy of your application on new infrastructure, you declaratively define the parameters of the experiment (10% of traffic to version Y, 90% of traffic to version X), and the service mesh implements it for you. Although, as a developer, you are involved in defining the experiment, the implementation is transparent and automatic, meaning that many more experiments will be run with a corresponding increase in reliability, agility, and insight.

Introspection

If you are like most programmers, once you write a program, you repeatedly debug it as new errors manifest themselves. Finding errors in your code is a large part of how most developers spend their days. Debugging is even more difficult when applications are spread across multiple microservices. It is hard to stitch together a single request when it spans multiple Pods. The information needed for debugging must be stitched back together from multiple sources, assuming that the relevant information was collected in the first place.

Automatic introspection is another important capability provided by a service mesh. Because it is involved in all communication between Pods, the service mesh knows where requests were routed, and it can keep track of the information required to put a complete request trace back together. Instead of seeing a flurry of requests to a bunch of different microservices, the developer can see a single aggregate request that defines

the user experience of their complete application. Furthermore, the service mesh is implemented once for an entire cluster. This means that the same request tracing works no matter which team developed the service. The monitoring data is entirely consistent across all of the different services joined together by a cluster-wide service mesh.

Do You Really Need a Service Mesh?

The advantages described here may have you eager to start installing a service mesh on your cluster. However, before you do that, it's worth considering whether a service mesh is really necessary for your application. A service mesh is a distributed system that adds complexity to your application design. The service mesh is deeply integrated into the core communication of your microservices. When a service mesh fails, your entire application stops working. Before you adopt a service mesh, you must be confident that you can fix problems when they occur. You must also be ready to monitor the software releases for the service mesh to make sure that you pick up the latest security and bug fixes, and, of course, when fixes become available, you must also be ready to roll out the new version without impacting your application. This additional operational overhead means that for many small applications, a service mesh is an unnecessary complexity.

If you are using Kubernetes provided as a managed service that also provides a service mesh, it is much easier to use that mesh, knowing that the cloud provider will provide the support, debugging, and seamless new releases for the service mesh. But even with a service mesh supplied by a cloud provider, there is additional complexity for your developers to learn. Ultimately, weighing the costs versus benefits of a service mesh is something each application or platform team needs to do at a cluster level. To maximize the benefits of a service mesh, it's helpful for all microservices in the cluster to adopt it at the same time.

Introspecting a Service Mesh Implementation

There are many different service mesh projects and implementations in the cloud native ecosystem, but most share many of the same design patterns and technology. Because the service mesh is transparently intercepting network traffic from your application Pod, modifying it and rerouting it through the cluster, a part of the service mesh needs to be present within every one of your Pods. Forcing developers to add something to each container image would introduce significant friction as well as make it much more difficult to centrally manage the service mesh version. As a result, most service mesh implementations add a sidecar container to every Pod in the mesh. Because the sidecar sits in

the same network stack as the application Pod, it can use tools like `iptables` or, more recently, `eBPF` to introspect and intercept network traffic coming from your application container and process it into the service mesh.

Of course, requiring every developer to add a container image to their Pod definition introduces nearly as much friction as requiring them to modify their container image. To address this, most service mesh implementations depend on a mutating admission controller to automatically add the service mesh sidecar to all Pods that are created in a particular cluster. Any REST API request to create a Pod is first routed to this admission controller. The service mesh admission controller modifies the Pod definition by adding the sidecar. Because this admission controller is installed by the cluster administrator, it transparently and consistently implements a service mesh for an entire cluster.

But the service mesh isn't just about modifying the Pod network. You also need to be able to control how the service mesh behaves; for example, by defining routing rules for experiments or access restrictions for services in the mesh. Like everything else in Kubernetes, these resource definitions are declaratively specified via JSON or YAML object definitions that you create using `kubectl` or other tools that communicate with the Kubernetes API server. Service mesh implementations take advantages of *custom resource definitions* (CRDs) to add specialized resources to your Kubernetes cluster that are not part of the default installation. In most cases, the specific custom resources are tightly tied to the service mesh itself. An ongoing effort in the CNCF is defining a standard vendor-neutral Service Mesh Interface (SMI) that many different service meshes can implement.

Service Mesh Landscape

The most daunting aspect of the service mesh landscape may be figuring out which mesh to choose. So far, no one mesh has emerged as the de facto standard. Though concrete statistics are hard to come by, the most popular service mesh is likely the Istio project. In addition to Istio, there are many other open source meshes, including Linkerd, Consul Connect, Open Service Mesh, and others. There are also proprietary meshes like AWS App Mesh. We expect efforts to standardize these interfaces to continue in the coming years in the cloud native community.

How is a developer or cluster administrator to choose? The truth is that the best service mesh for you is likely the one that your cloud provider supplies for you. Adding operating a service mesh to the already complicated duties of your cluster operators is generally unnecessary. It is much better to let a cloud provider manage it for you.

If that isn't an option for you, do your research. Don't be drawn in by flashy demos and promises of functionality. A service mesh lives deep within your infrastructure, and any failures can significantly impact the availability of your application. Additionally, because service mesh APIs tend to be implementation specific, it is difficult to change your choice of service mesh once you have spent time developing applications around it. In the end, you may find that the right mesh for you is no mesh at all.

Summary

Service meshes contain powerful functionality that adds security and flexibility to your application. At the same time, a service mesh adds complexity to the operations of your cluster and is another potential source of outages for your application. Carefully consider the pros and cons of adding service meshes to your infrastructure. If you have the choice, use a managed service mesh where someone else takes responsibility for the operational details while enabling your applications access to service mesh capabilities.

Previous chapter
14. Role-Based Access Control for Kubernetes

Next chapter
16. Integrating Storage Solutions and Kubernetes